

- `forloop.first`—A Boolean that is `True` for the first iteration
- `forloop.last`—A Boolean that is `True` for the last iteration
- `forloop.parentloop`—The `forloop` object for the surrounding loop, in the case of nested loops

3.5.3 Comment blocks

There are two ways to write a comment in the Django template language: inline and using a multiline tag. *Inline comments* use matching `{#` and `#}` braces, while *multiline comments* use a `{% comment %}` block tag. The multiline form is useful for temporarily commenting out parts of a template while you are debugging. This code has an inline comment beside `Guitar` and has commented out the `Oboe` and `Bassoon` tags:

```
<ul>
  <li> Guitar {# cool instrument #} </li>
  <li> Drums </li>
  {% comment "For orchestras only" %}
  <li> Oboe </li>
  <li> Bassoon </li>
  {% endcomment %}
</ul>
```

When the preceding HTML gets rendered, the opinion next to `Guitar` is not shown. Likewise, neither of the symphony instruments appear in the list. The `{% comment %}` tag takes an optional argument of a string used as a note about the comment. Yes, you can comment your comment. It might seem silly, but if you are commenting out a large block of a template, it is sometimes helpful to leave yourself a note reminding yourself why.

3.5.4 Verbatim blocks

One thing almost every coder enjoys writing about is coding. Many characters that are meaningful in HTML you will also find in your code. This can make for some messy HTML. Writing a Python `if` statement with comparison operators in a web page requires entities like `>` and `<` to display properly.

This problem is even worse if you're writing a website about Django or JavaScript templating. Both of these cases use characters that are special to HTML or the Django Template Engine itself. There is a tag that solves this: `{% verbatim %}`. This block tells the template engine to render everything contained within exactly how it is. You can even name a `{% verbatim %}` block so that you can wrap `{% verbatim %}` blocks. The following code contains `{{ }}` operators and a `{% verbatim %}` tag that are ignored by the Django Template Engine, as they're all inside a `{% verbatim %}` block:

```
{% verbatim myblock %}
  Many JavaScript templates use {{ }} operators which overlap
  with the Django Template Engine. A similar problem exists if
  you wish to show a tag like {% verbatim %}.
{% endverbatim myblock %}
```